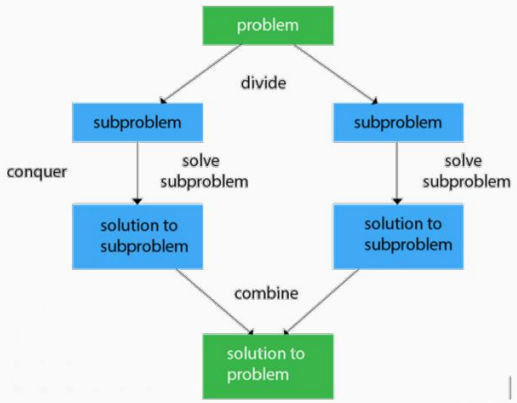
7 basic principle of object orientation.

Principle 1: Divide and Conquer

- The **first step** in designing a program is to **divide** the overall program into a number of objects
- The objects will interact with each other to solve the problem
- Problem-solving: Break problems (programs) into **small, manageable tasks**

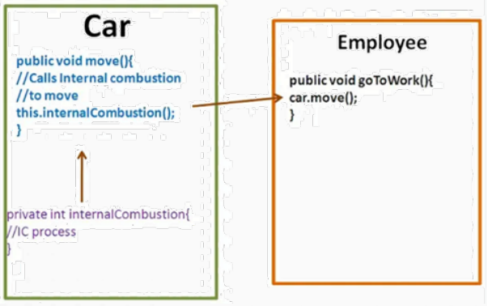


Source: https://www.tpointtech.com/divide-and-conquer-introduction

Principle 2: Encapsulation& Modularity

Principle #2: Encapsulation & Modularity

- The next step in designing a program is to decide for **each object**, what **attribute** it has and what **actions** it will take.
- The goal is that each object is a **self-contained module** with a clear responsibility and the attributes and actions necessary to carry out its role.
- Problem-solving: Each object knows how to **solve its task** and **has the information** it needs.

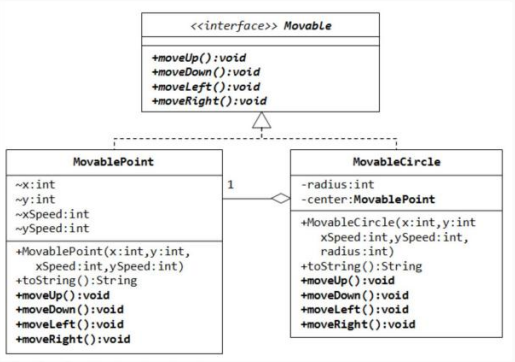


Source: https://www.dineshonjava.com/encapsulation-in-java/

Principle 3:Public Interface

Principle #3: Public Interface

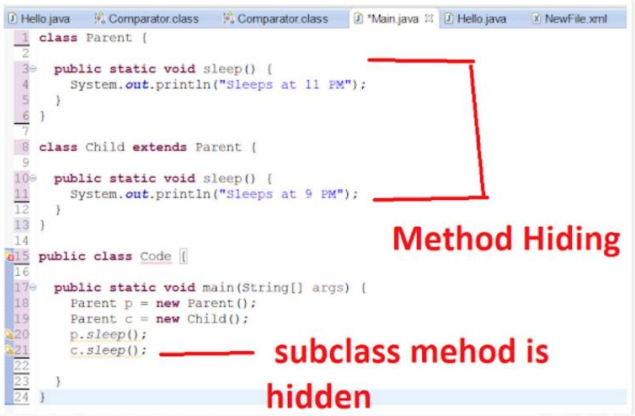
- For **objects** to work cooperatively and efficiently, clarify exactly how they should **interact, or interface**, with one another.
- Each object should present a **clear public interface** that determines how other objects will be used.



Source: https://stackoverflow.com/questions/28520164/why-are-interfaces-helpful

Principle 4: Information Hiding

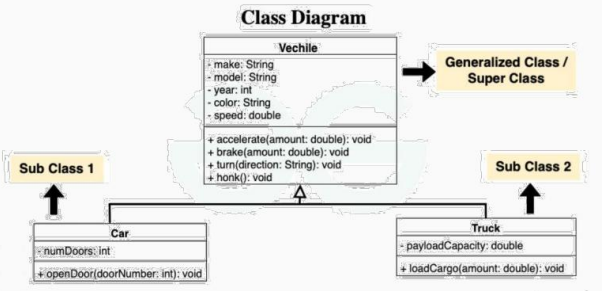
- To enable objects to work together cooperatively, **certain details** of their design and performance should be **hidden** from other objects.
- Each object should **shield** its users from **unnecessary details** of how it performs its role.



Source: https://www.java67.com/2019/07/java-variable-and-method-hiding-example.html#google_vignette

Principle 5: Generality

- To make an object as generally useful as possible, design them **not for a particular task but** for a **particular kind of task**.
- This principle underlies the use of software libraries.
- Objects should be designed to be as **general** as possible.
- Objects are designed to solve a kind of task rather than a singular task.

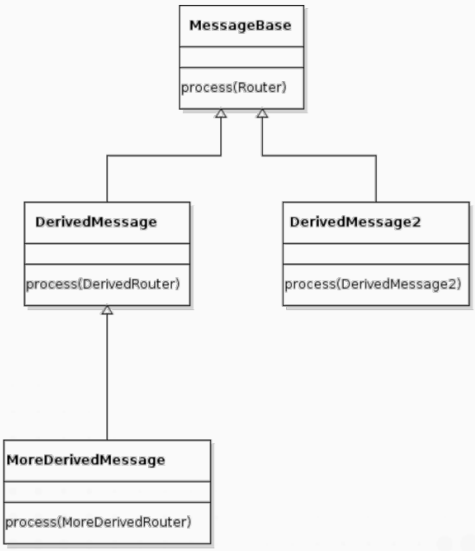


Source: https://www.geeksforgeeks.org/generalization-in-ooad/

Principle 6: Extensibility

Principle #6: Extensibility

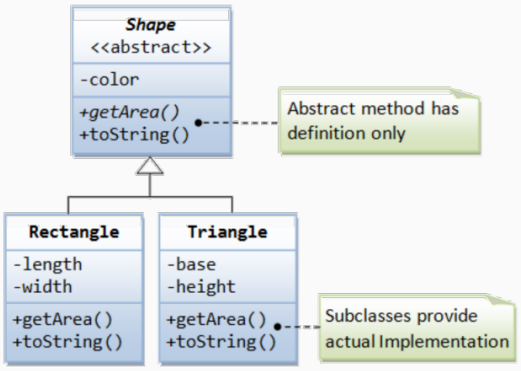
- One of the strengths of the OOP approach is the ability to extend an **object's behavior** to handle new tasks.
- An object should be designed so that its functionality **can be extended** to carry out more specialized tasks.



Source: https://stackoverflow.com/questions/28902349/two-way-extensible-hierarchy-with-java

Principle 7: Abstarction

- Abstraction is the ability to **focus on the important features** of an object when trying to work with large amounts of information.
- The objects in Java programs will be abstractions in this sense because they **ignore many of the attributes** that characterize the real objects
- They **focus** only on those **attributes that are essential** for solving a particular problem.



Source:https://stackoverflow.com/questions/28038173/how-do-we-achieve-abstraction-using-interfaces-in-java

Benefits of OOP

- Save development time and cost by **reusing code**
  - Once an object class is created it can be used in other applications
- Easier **debugging**:
  - Classes can be tested independently
  - Reused objects have already been tested